

Introduction to PowerBI & Power Query

Hands-On Tutorial: Connecting to Data in Three Ways

Pedro Cardoso

M.Sc. in Electrical and Computer Engineering
Instituto Superior de Engenharia (ISE), Universidade do Algarve
pcardoso@ualg.pt | sites.google.com/view/pedrojscardoso

Last updated: May 12, 2026

Project 2.1 – Connecting to Data

M.Sc. in Electrical and Computer Engineering – ISE – Universidade do Algarve

Scenario

In this tutorial you will import the same Titanic passenger data into Power BI using three different routes:

- direct CSV import,
- SQLite through a Python script,
- a local REST API through the Web connector.

1 Learning Outcomes

By the end of this tutorial, you will be able to:

- identify the main categories inside **Home** → **Get Data**,
- load structured data from files, Python, and web APIs,
- perform basic Power Query cleaning on a real CSV,
- compare three ingestion routes for the same dataset.

2 Project Files

- Dataset: `Data/Titanic.csv`
- SQLite builder: `scripts/prepare_titanic_db.py`
- Flask API: `scripts/api.py`

3 Prerequisites

- Power BI Desktop installed.
- Python 3.10 or later installed.
- Python packages: pandas and flask.

Install the Python dependencies:

```
pip install -r scripts/requirements.txt
```

4 Lab Workflow Overview

1. Inspect the **Get Data** categories in Power BI.
2. Load Titanic data from CSV.
3. Clean the CSV in Power Query.
4. Build a SQLite database from the same dataset.
5. Load the SQLite data with the Power BI Python connector.
6. Start a Flask API and load the same data with the Web connector.

5 Step 1 – Review the Get Data Taxonomy

Open **Home** → **Get Data** and identify the following source groups:

- **File:** CSV, Excel, JSON, XML, PDF, Parquet.
- **Database:** SQL Server, PostgreSQL, MySQL, Oracle.
- **Power Platform:** Dataverse, Power BI datasets.
- **Azure:** Azure SQL, Data Lake, Blob storage.
- **Online Services:** SharePoint, GitHub, Google Analytics.
- **Other:** Web, OData, Python script, R script, Blank query.

6 Step 2 – Route A: Load the CSV

1. Open **Power BI Desktop**.
2. Select **Home** → **Get Data** → **Text/CSV**.
3. Browse to `Data/Titanic.csv`.
4. Confirm the preview and click **Transform Data**.

Important

Do not click **Load** directly. CSV files often need cleaning before they are safe to analyse.

7 Step 3 – Clean the CSV in Power Query

The Titanic CSV uses semicolons as delimiter and includes a mix of decimal formats.

1. Check that the separator is interpreted correctly.
2. Review the data types of all columns.
3. Apply or confirm these transformations:
 - sibsp, parch, body: Whole Number
 - age, fare: Decimal Number
 - sex, name, ticket, cabin, boat, home.dest: Text
4. Replace the embarkation codes as follows:
 - S → Southampton, England
 - C → Cherbourg, France
 - Q → Queenstown, Ireland
5. Replace the survived codes to:
 - 0 → Died
 - 1 → Survived
6. Replace the pclass codes to:
 - 1 → 1st Class
 - 2 → 2nd Class
 - 3 → 3rd Class
7. Add a custom column named AgeGroup:


```
= if [Age] = null then "Unknown"  
  else if [Age] < 15 then "Child"  
  else if [Age] < 60 then "Adult"  
  else "Senior"
```
8. Click **Close & Apply**.

Power Query M – Reference Implementation (inspect via Home → Advanced Editor; adjust the path in File . Contents to your local folder)

```

let
    Source = Csv.Document(
        File.Contents("Data/Titanic.csv"),
        [Delimiter=";", Columns=14,
        Encoding=65001, QuoteStyle=QuoteStyle.None]),

    PromotedHeaders = Table.PromoteHeaders(Source,
        [PromoteAllScalars=true]),

    ChangedTypes = Table.TransformColumnTypes(PromotedHeaders,
        {"sibsp", Int64.Type},
        {"parch", Int64.Type},
        {"body", Int64.Type})),

    // survived: 1 -> Survived, 0 -> Died
    SurvivedLabel = Table.ReplaceValue(ChangedTypes,
        "1", "Survived",
        Replacer.ReplaceText, {"survived"}),
    DiedLabel = Table.ReplaceValue(SurvivedLabel,
        "0", "Died",
        Replacer.ReplaceText, {"survived"}),

    // embarked: code -> full city name
    Southampton = Table.ReplaceValue(DiedLabel,
        "S", "Southampton, England",
        Replacer.ReplaceText, {"embarked"}),
    Cherbourg = Table.ReplaceValue(Southampton,
        "C", "Cherbourg, France",
        Replacer.ReplaceText, {"embarked"}),
    Queenstown = Table.ReplaceValue(Cherbourg,
        "Q", "Queenstown, Ireland",
        Replacer.ReplaceText, {"embarked"}),

    // pclass: 1/2/3 -> 1st/2nd/3rd class
    Class1 = Table.ReplaceValue(Queenstown,
        "1", "1st class",
        Replacer.ReplaceText, {"pclass"}),
    Class2 = Table.ReplaceValue(Class1,
        "2", "2nd class",
        Replacer.ReplaceText, {"pclass"}),
    Class3 = Table.ReplaceValue(Class2,
        "3", "3rd class",
        Replacer.ReplaceText, {"pclass"}),

    // age groups: Child < 15, Adult < 60, Senior >= 60
    AgeGroup = Table.AddColumn(Class3, "Custom", each if [age] = null then "Unknown"
        " else if [age] < 15 then "Child" else if [age] < 60 then "Adult" else "
        Senior")
in
    AgeGroup

```

8 Step 4 – Route B: Build SQLite from Python

Open a terminal in the project root and run:

```
python scripts/prepare_titanic_db.py
```

This command creates Data/titanic.db with one table named passengers.

Checkpoint

Before continuing, verify that `Data/titanic.db` now exists.

9 Step 5 – Load SQLite via the Power BI Python Script Connector

Power BI has no native SQLite connector, so we use Python as a bridge.

1. Select **Home** → **Get Data** → **Other** → **Python script**.
2. Paste the following code:

```
import sqlite3
import pandas as pd

conn = sqlite3.connect(r"Data/titanic.db")
passengers = pd.read_sql("SELECT * FROM passengers", conn)
conn.close()
```

3. Click **OK**.
4. In Navigator, select the `passengers` table.
5. Click **Transform Data** or **Load**.

10 Step 6 – Route C: Expose the Data Through a REST API

In a terminal, start the Flask API:

```
python scripts/api.py
```

By default, the API runs at:

```
http://127.0.0.1:5000/passengers
```

Optional test in a browser:

```
http://127.0.0.1:5000/health
```

11 Step 7 – Load the API in Power BI

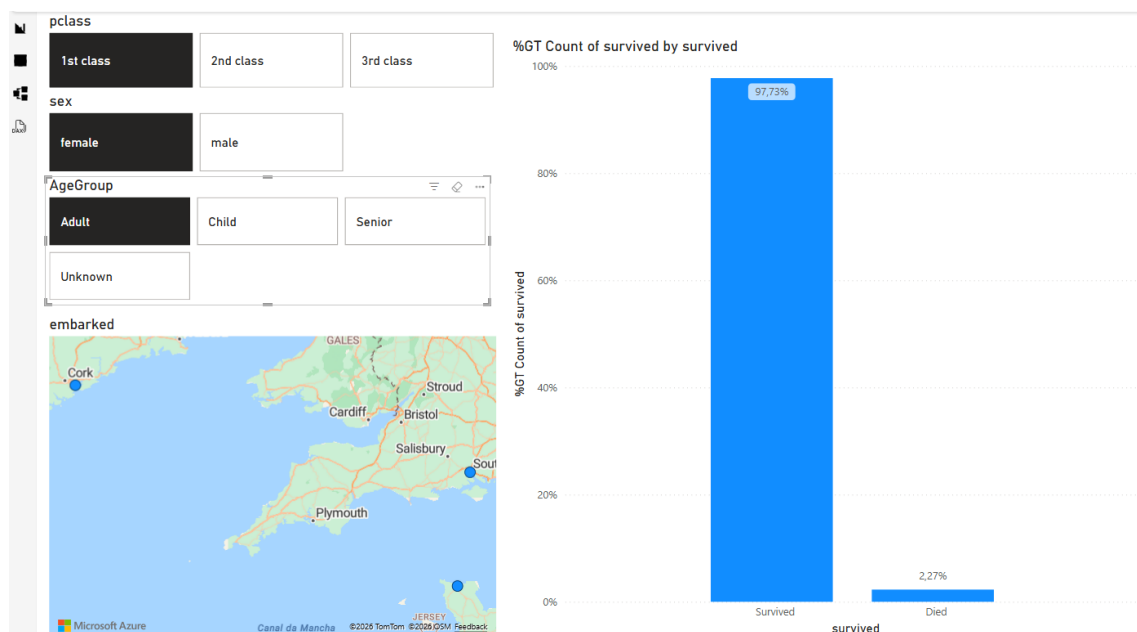
1. Select **Home** → **Get Data** → **Web**.
2. Enter:

```
http://127.0.0.1:5000/passengers
```

3. Click **OK**.

4. In Power Query, expand the JSON list into a table.
5. Apply any final type adjustments and click **Close & Apply**.

Reference Report View The screenshot below shows an example of a completed report canvas using filters, map, and a survival chart.



12 Mini Analysis Task

After testing the three loading routes, create at least two visuals using one of the imported tables:

- a bar chart of passenger count by class,
- a chart comparing survivors and non-survivors.

13 Reflection Questions

Answer the following in short text:

1. Which route was the fastest to load?
2. Which route gives the most flexibility for automation or integration?
3. Which route would you recommend for a one-off classroom analysis? Why?
4. Which route would you recommend for a reusable data pipeline? Why?

14 Expected Takeaways

- **CSV** is the simplest route, but often requires the most manual cleaning.
- **Python + SQLite** is useful when a source is not supported directly by Power BI.
- **REST API** is the most flexible route for connected systems and repeatable integrations.